

SRS (Virtual Bookstore Project)

1. Introduction

The Virtual Bookstore is a microservices-based web application designed to provide a secure, scalable, and user-friendly digital platform for browsing books, managing shopping carts, placing orders, processing payments, and submitting reviews. The backend is divided into independent services for authentication, catalog, cart, order, payment, and reviews, with service discovery handled by Eureka and request routing handled by an API Gateway.

2. Project Scope

The scope of the project includes guest browsing, user authentication, catalog management, shopping cart management, checkout, order management, Stripe payment processing, and review management. Administrative capabilities include managing books, categories, user accounts, order statuses, and stock monitoring. The system also supports low-stock visibility for inventory control.

2.1 In Scope

Guest browsing of books and book details.

User registration and login with JWT-based authentication.

Book and category management.

Shopping cart operations: add, update, remove, clear.

Checkout and order history.

Stripe-based payment processing.

Review creation, update and deletion

Admin user management, role updates, and enabling/disabling users.

2.2 Out of Scope

Native mobile applications.

Physical delivery tracking and logistics integration.

Payment simulation or mock payment, because the current system integrates with Stripe.

External banking API integrations beyond the implemented Stripe flow.

3. Overall Description

3.1 Product Perspective

The Virtual Bookstore is a distributed web application where the Angular frontend communicates with backend microservices through the API Gateway. The services register with Eureka, communicate through Feign clients, and persist data in PostgreSQL(supabase) databases. The system is also containerized using Docker and docker-compose.

3.2 Product Functions

The system provides the following main functions:

Authentication and authorization.

Catalog and stock management.

Persistent shopping cart handling.

Order checkout and history.

Stripe payment handling.

Review management.

User and role administration.

3.3 User Classes and Characteristics

The system identifies the following user classes:

Guest: can browse books and view book details.

Registered User: can manage cart, place orders, pay, and submit reviews.

Admin: can manage books, categories, users, and orders.

Publisher: can be associated with books through the publisher reference in the catalog service.

3.4 Operating Environment

Frontend: Angular web application.

Backend: Java 21 / Spring Boot microservices.

Database: PostgreSQL.

Deployment: Docker and docker-compose.

Browsers: modern web browsers.

3.5 Design and Implementation Constraints

Passwords must not be stored in plain text; the authentication service uses password encoding and use hash function.

Authentication is token-based using JWT.

Stock must remain consistent after checkout and payment flows.

The system follows a layered architecture with controllers, services, repositories, DTOs, and Feign clients.

Cross-cutting concerns such as performance logging are handled with AOP.

4. Functional Requirements

4.1 Authentication and User Management

The system shall allow a new user to register with username, email, password, full name, and address. The system shall allow a registered user to log in and receive a JWT token.

The system shall allow admins to update user roles and enable/disable accounts.

4.2 Catalog Management

The system shall allow books to be created, updated, deleted, and viewed.

The system shall allow books to be filtered by category.

The system shall allow stock quantity to be checked and reduced.

The system shall allow low-stock books to be retrieved.

4.3 Shopping Cart

The system shall allow a registered user to add a book to the cart.

The system shall allow a user to update quantity, remove an item, and clear the cart.

The system shall calculate and preserve item prices in the cart.

The system shall ensure that the requested book exists and that enough stock is available before adding it to the cart.

4.4 Order Management

The system shall allow a user to checkout the current cart and generate an order.

The system shall calculate the total price of the order.

The system shall reduce stock after successful checkout.

The system shall allow users to view order history and order details.

The system shall allow admins to update order status.

4.5 Payment Processing

The system shall allow the user to initiate payment through Stripe.

The system shall create a payment record before redirecting to Stripe checkout.

The system shall update payment status when Stripe sends webhook events.

The system shall update the related order status after successful payment.

4.6 Reviews

The system shall allow users to create reviews for existing books.

The system shall allow users to update and delete their reviews.

The system shall allow users to view reviews by book.

5. Non-Functional Requirements

5.1 Security

Passwords shall be stored using password encoding, not plain text.

Access to sensitive operations shall be controlled using JWT-based authentication and role-based authorization.

Payment secrets and other sensitive configuration values shall be externalized through environment variables or configuration files.

5.2 Performance

The system should return catalog and search-related responses quickly under normal load.

The system should support multiple concurrent users through the microservices architecture and containerized deployment.

5.3 Usability

The user interface shall be simple and intuitive.

The web application shall be accessible through modern browsers.

5.4 Availability and Reliability

The system should remain available even if one service is restarted, because each microservice is independently deployed.

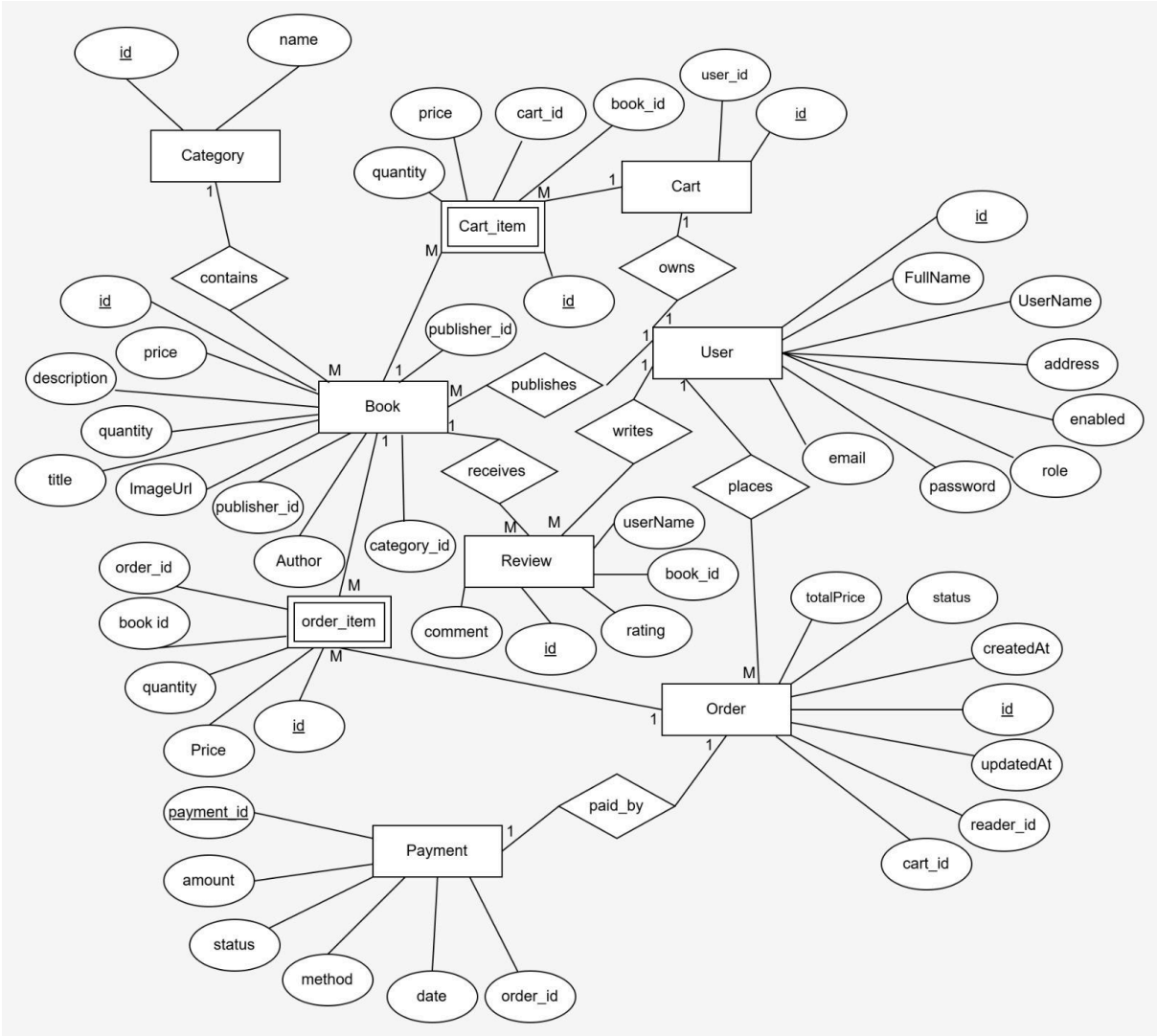
Data integrity must be maintained across checkout, order, and payment flows.

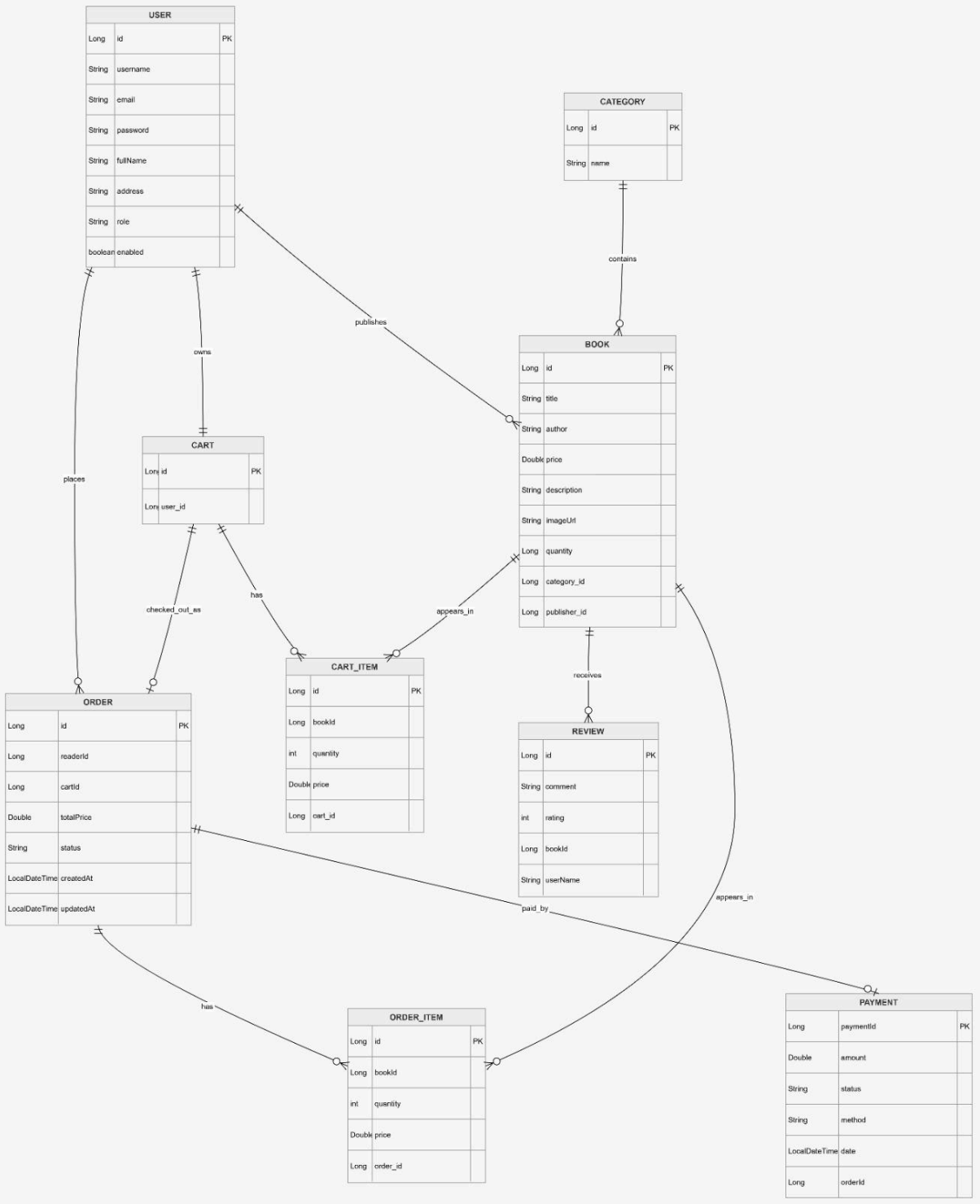
5.5 Maintainability

The codebase shall remain modular by service and by layer.

DTOs, Feign clients, AOP logging, and service separation shall be used to keep the system maintainable.

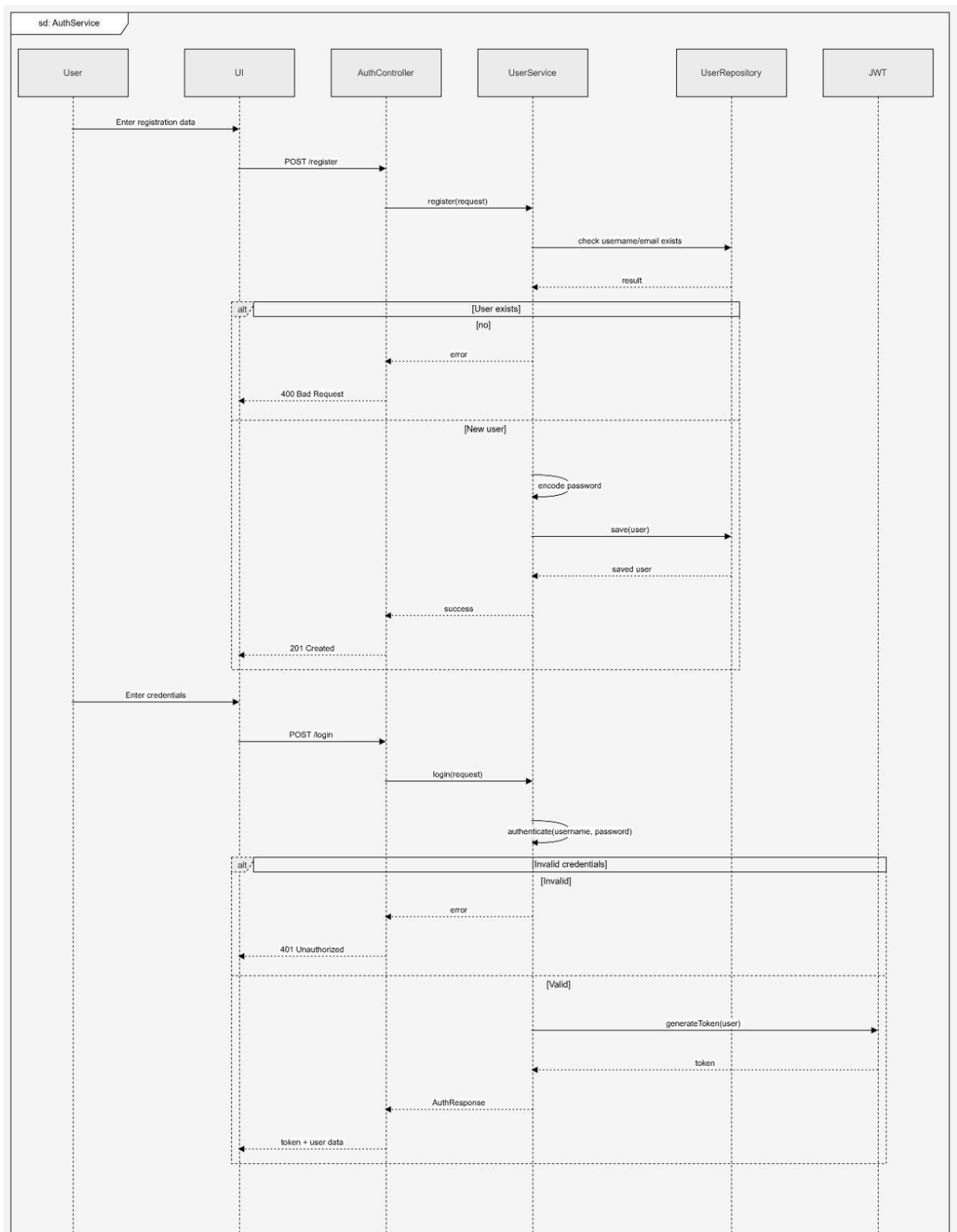
ERDs



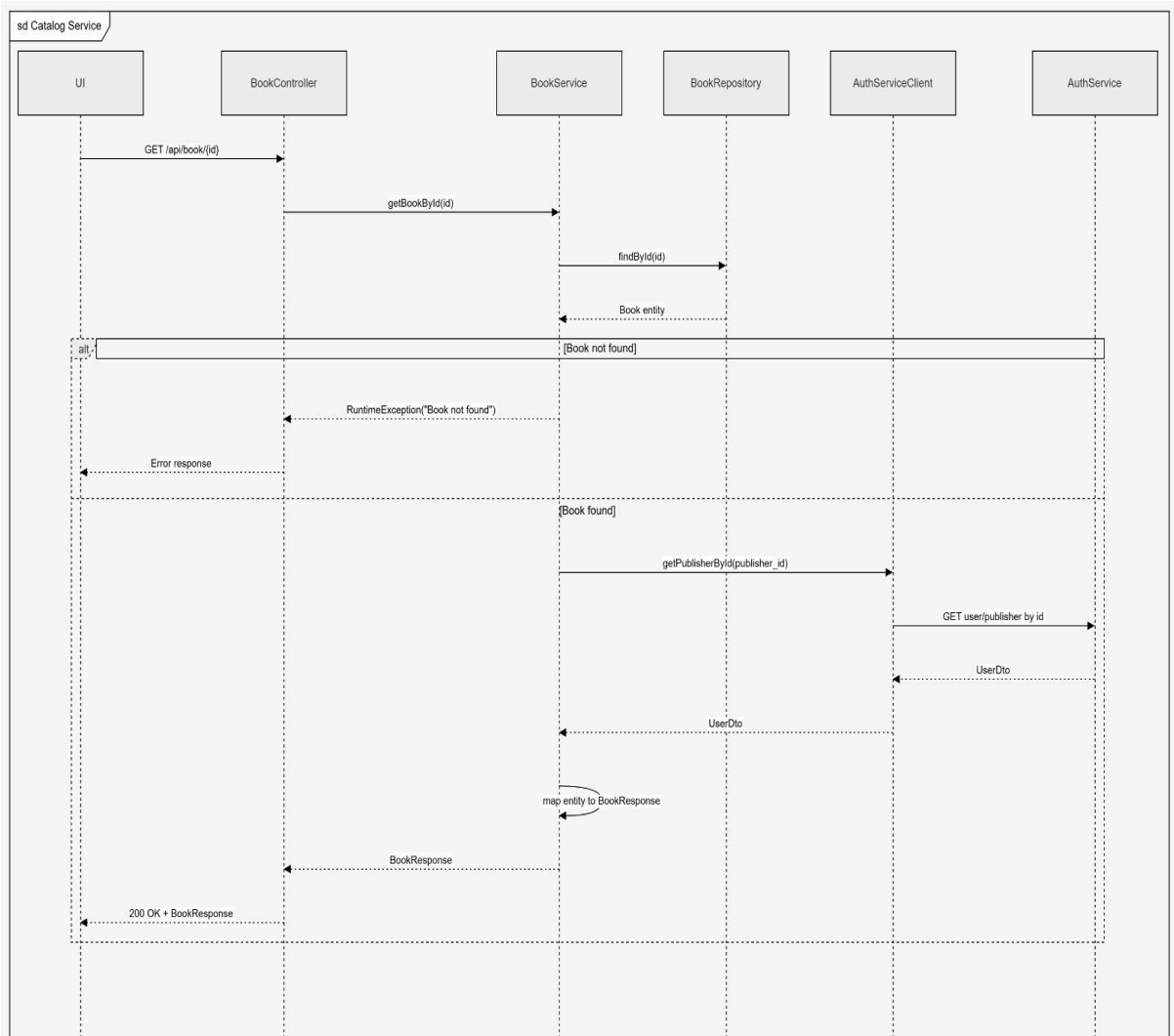


Sequence Diagrams

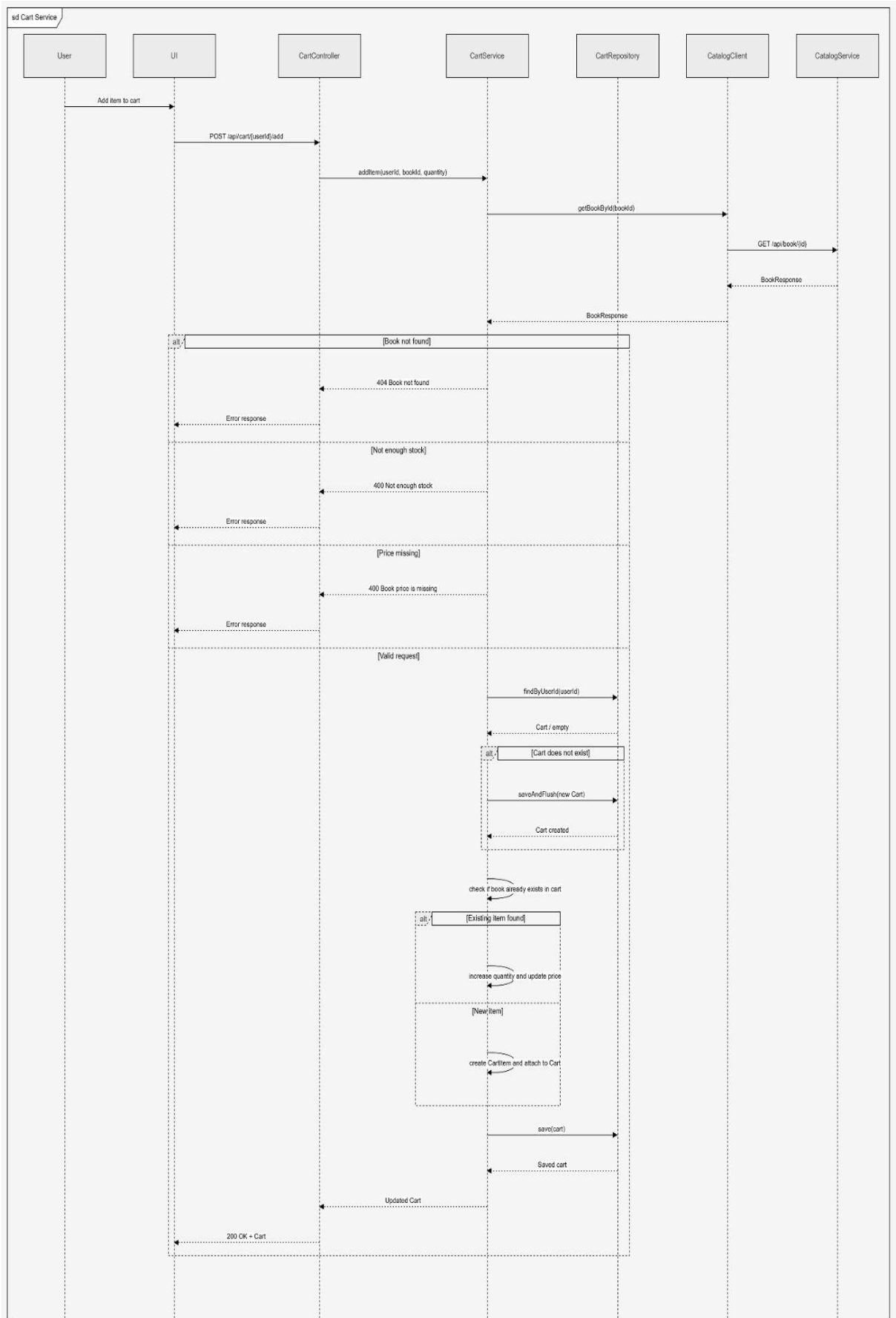
1) Authentication Service



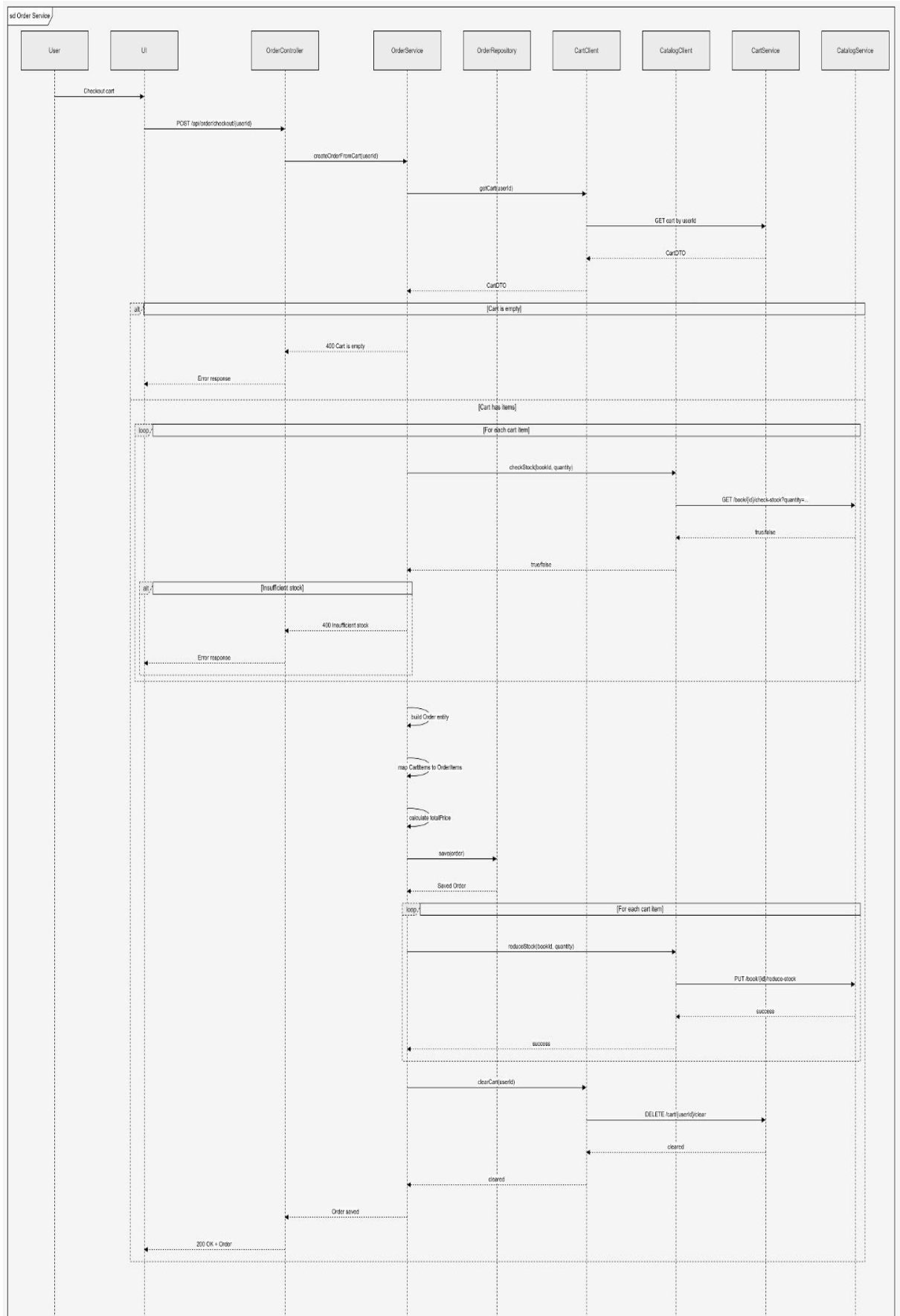
2) Catalog Service



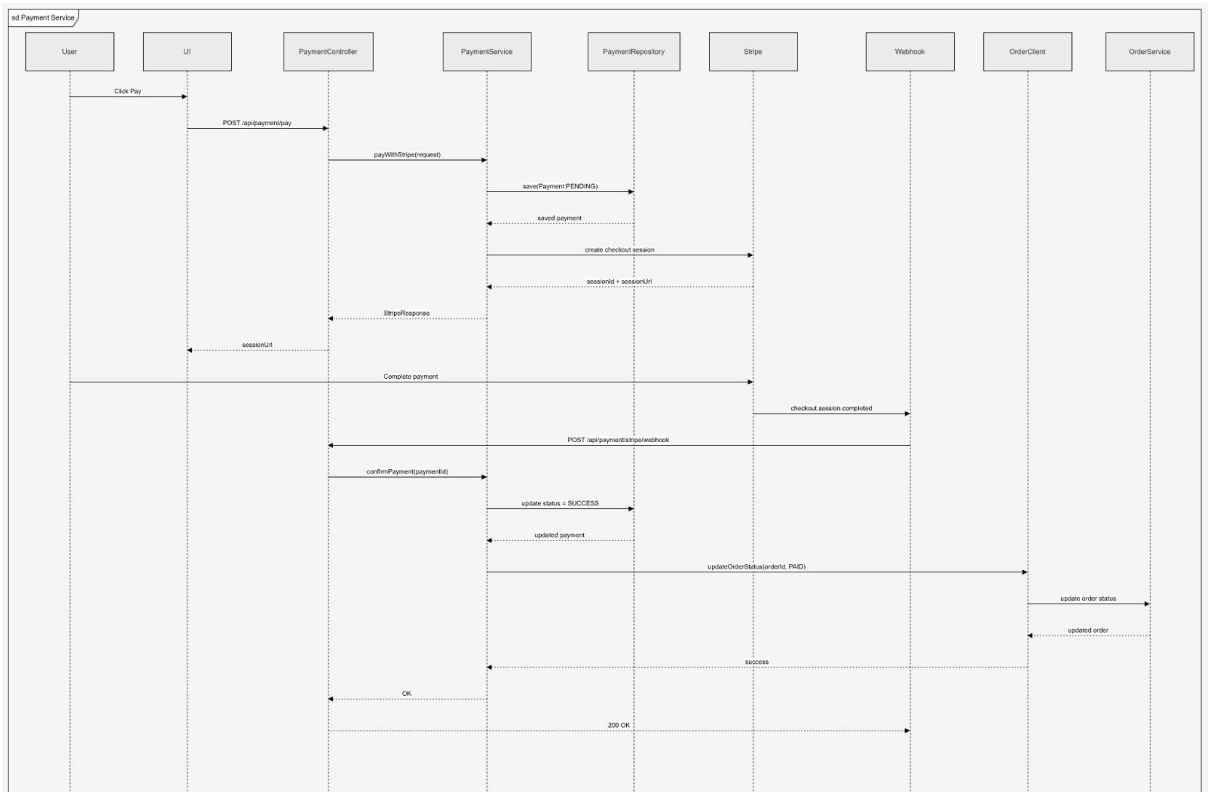
3) Cart Service



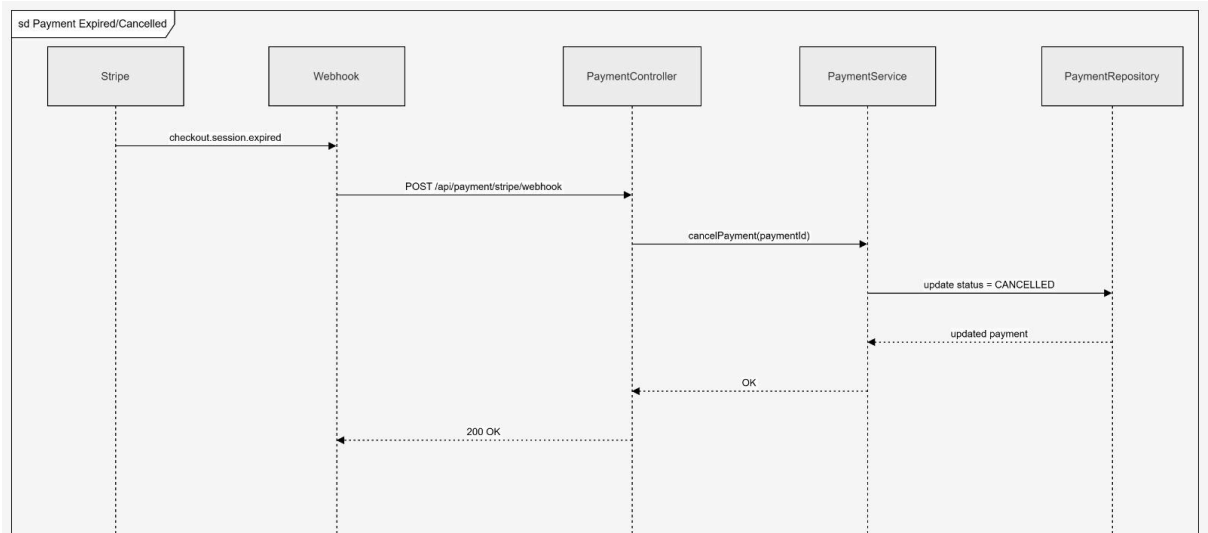
4) Order Service



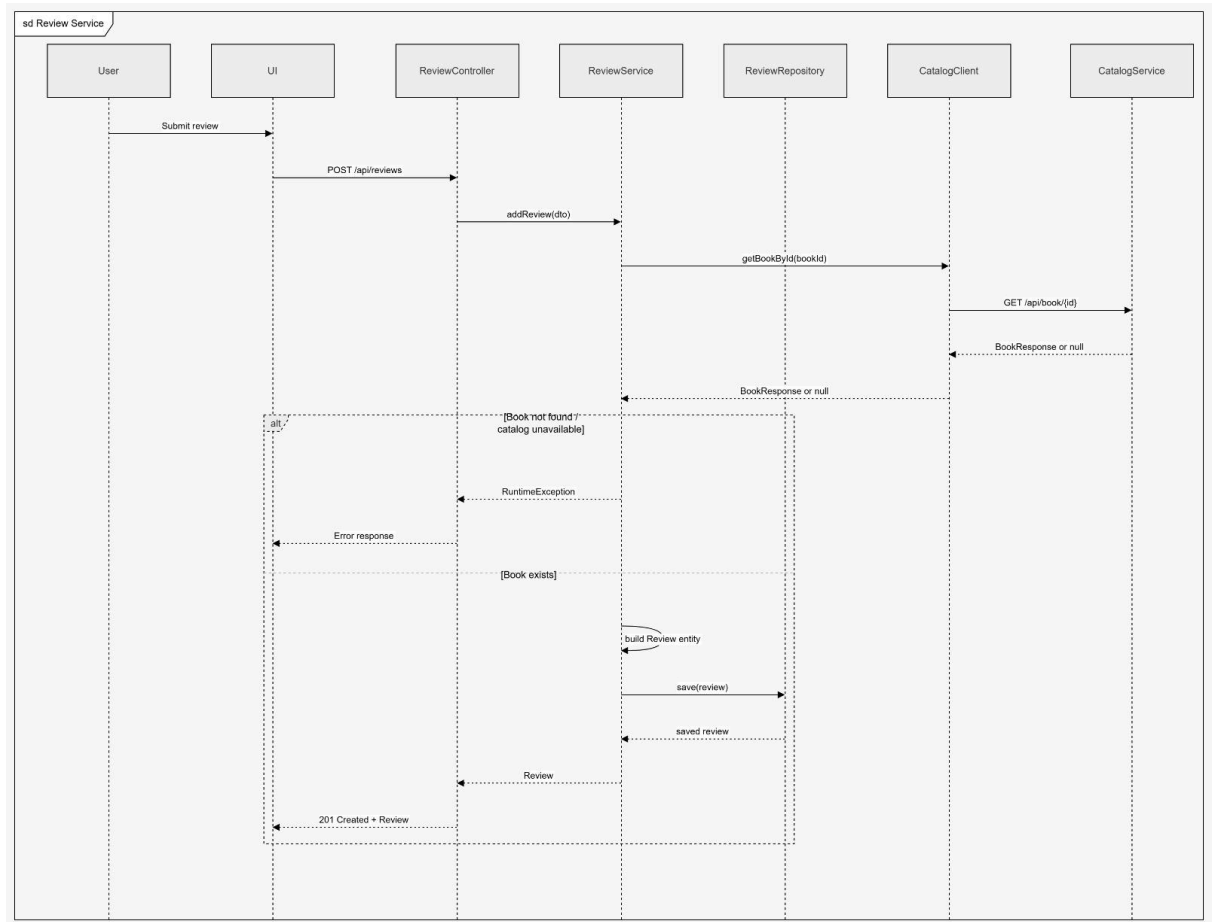
5) A- Payment Service



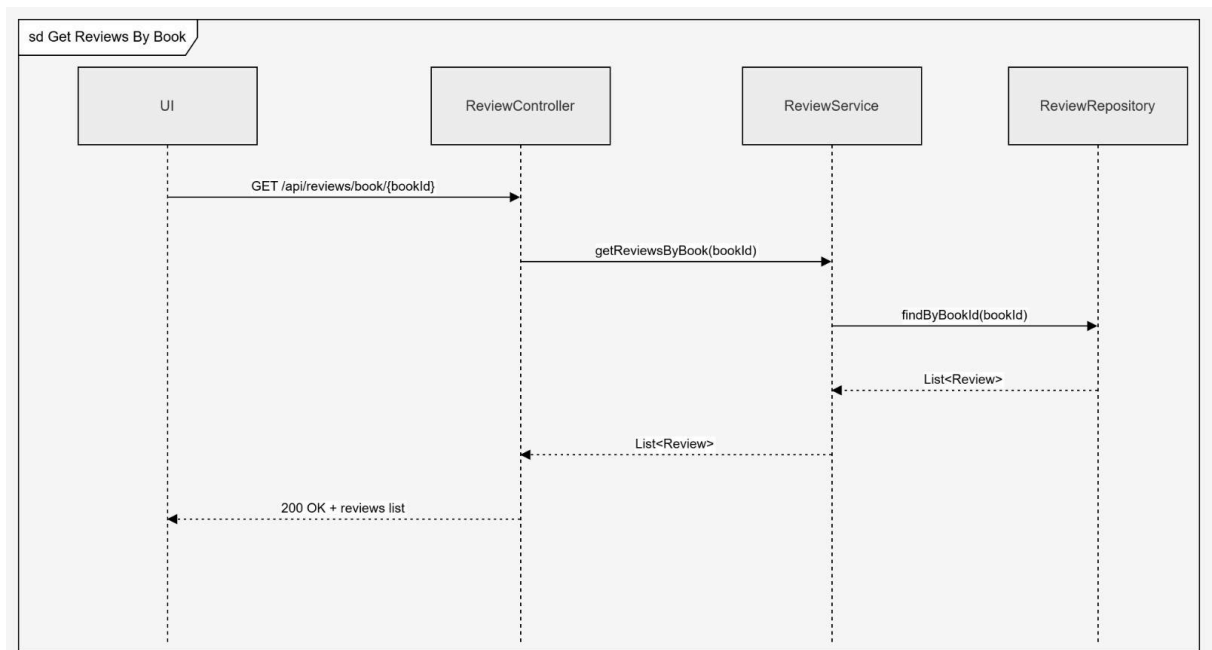
B- Payment Expired / Cancelled



6) A- Review Service

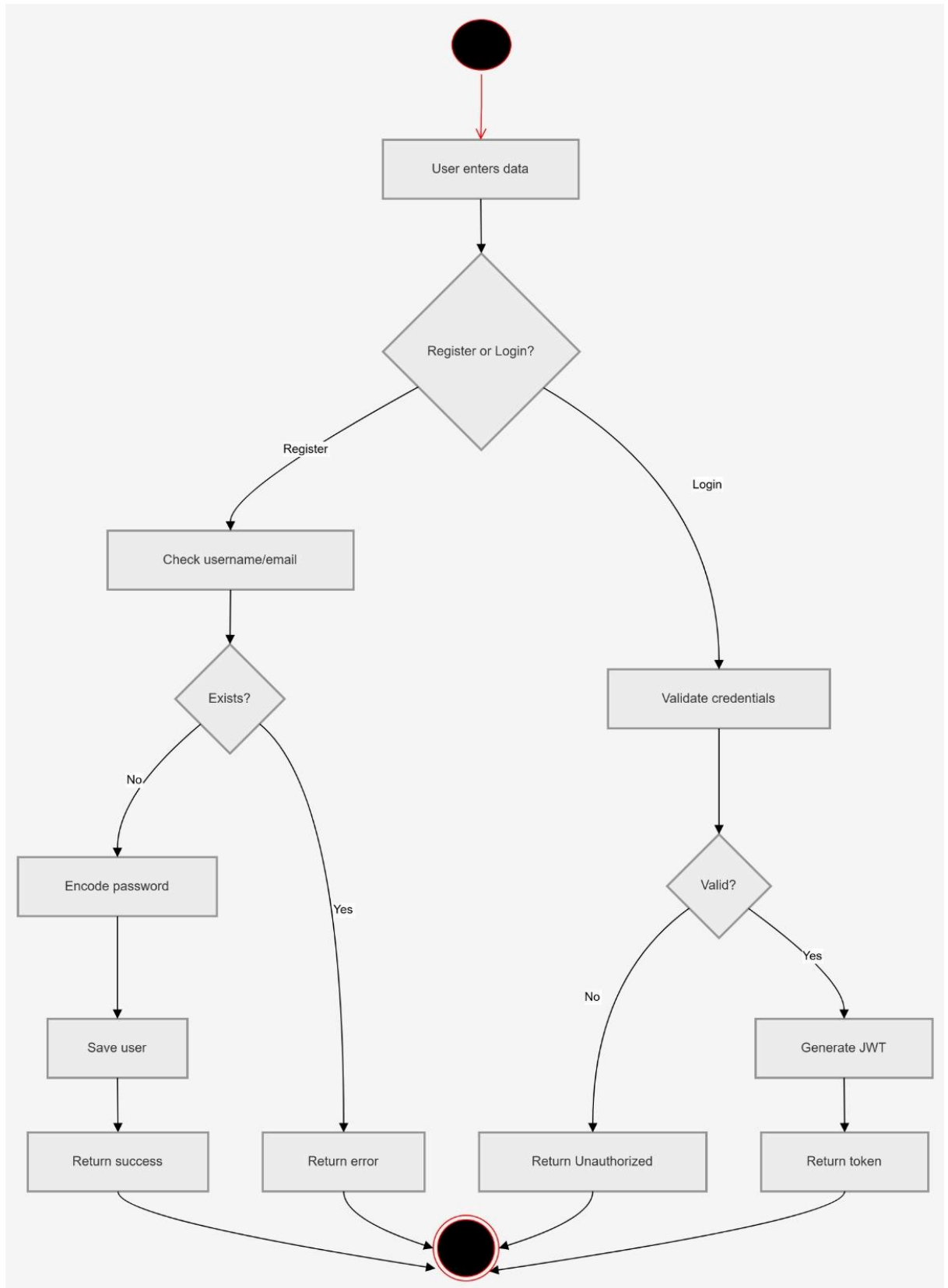


B- Get Reviews By Book

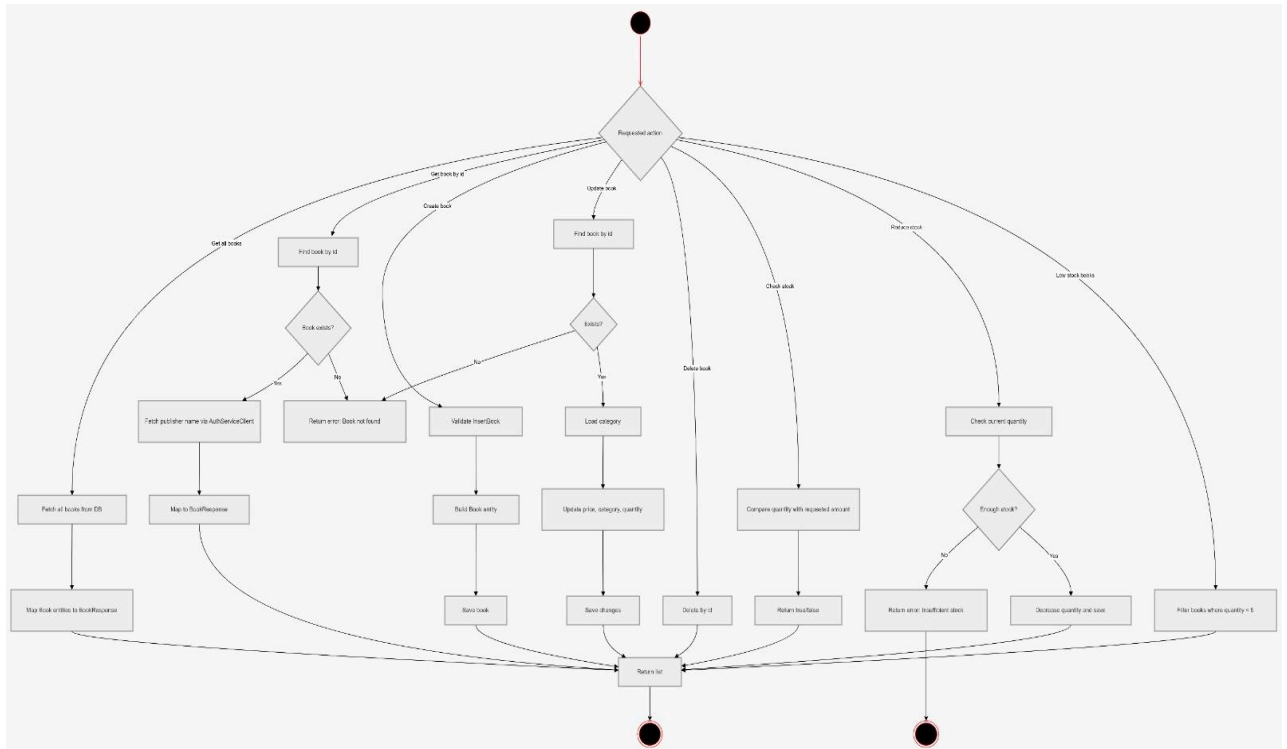


Activity Diagrams

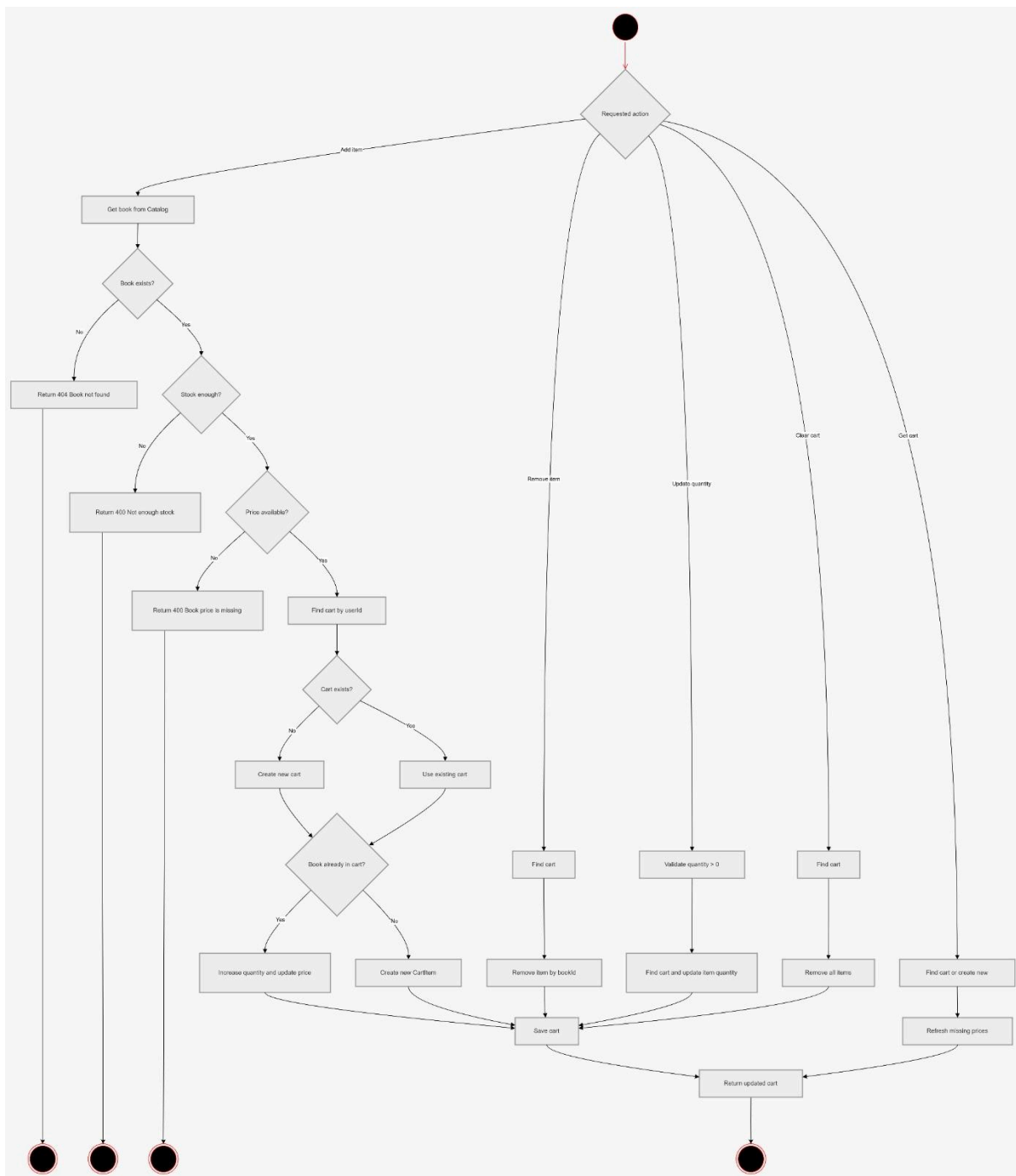
1) Authentication Service



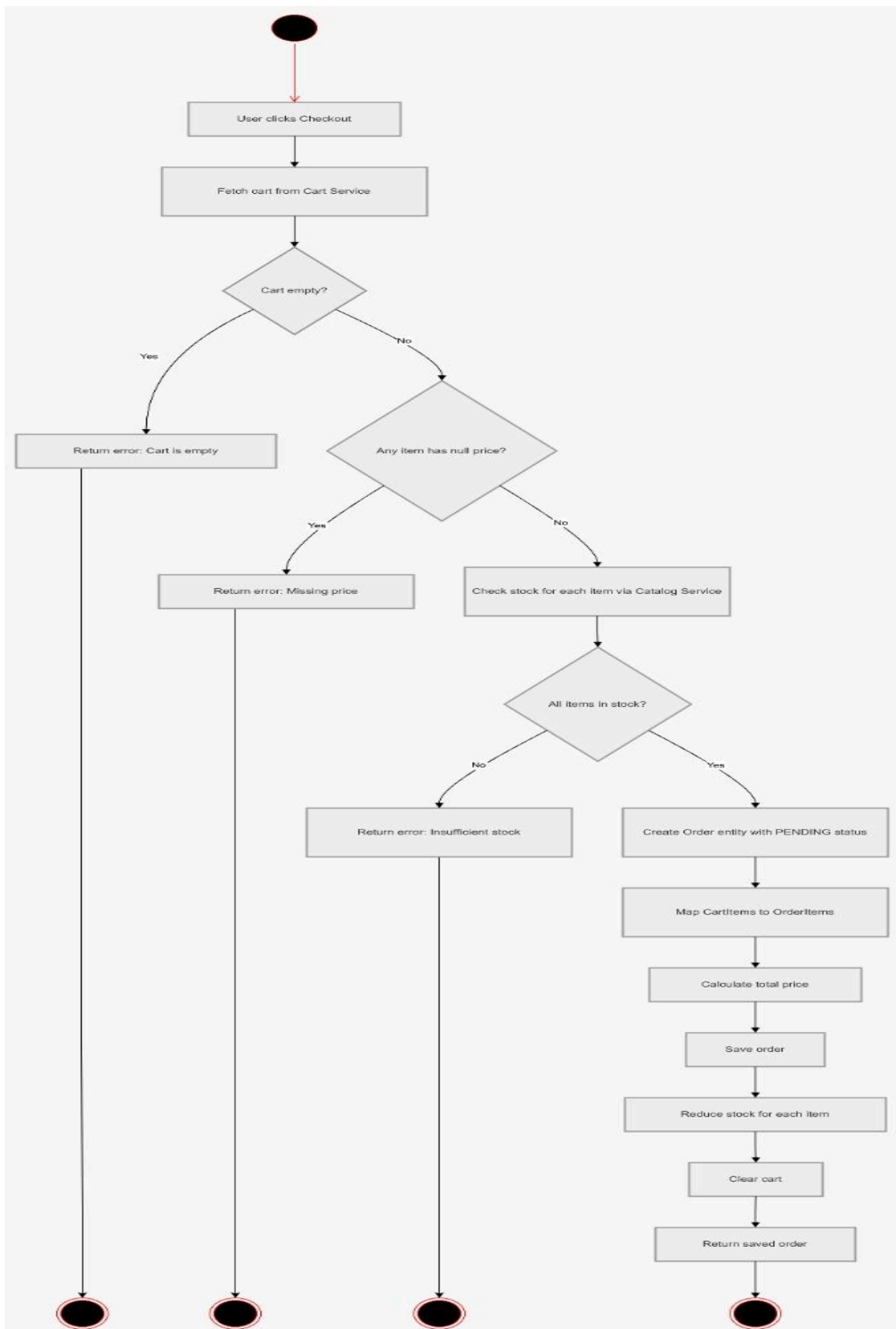
2) Catalog Service



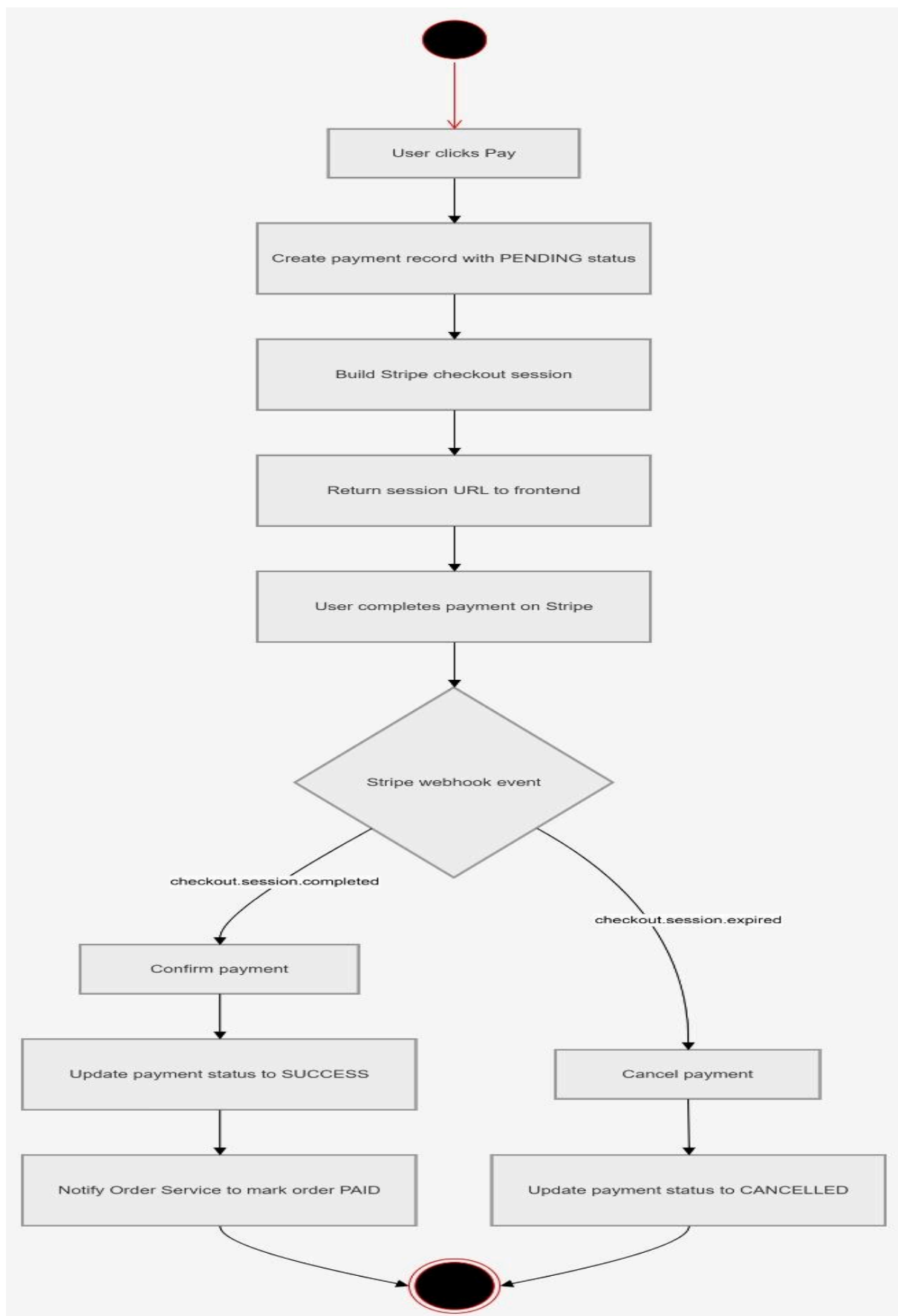
3) Cart Service



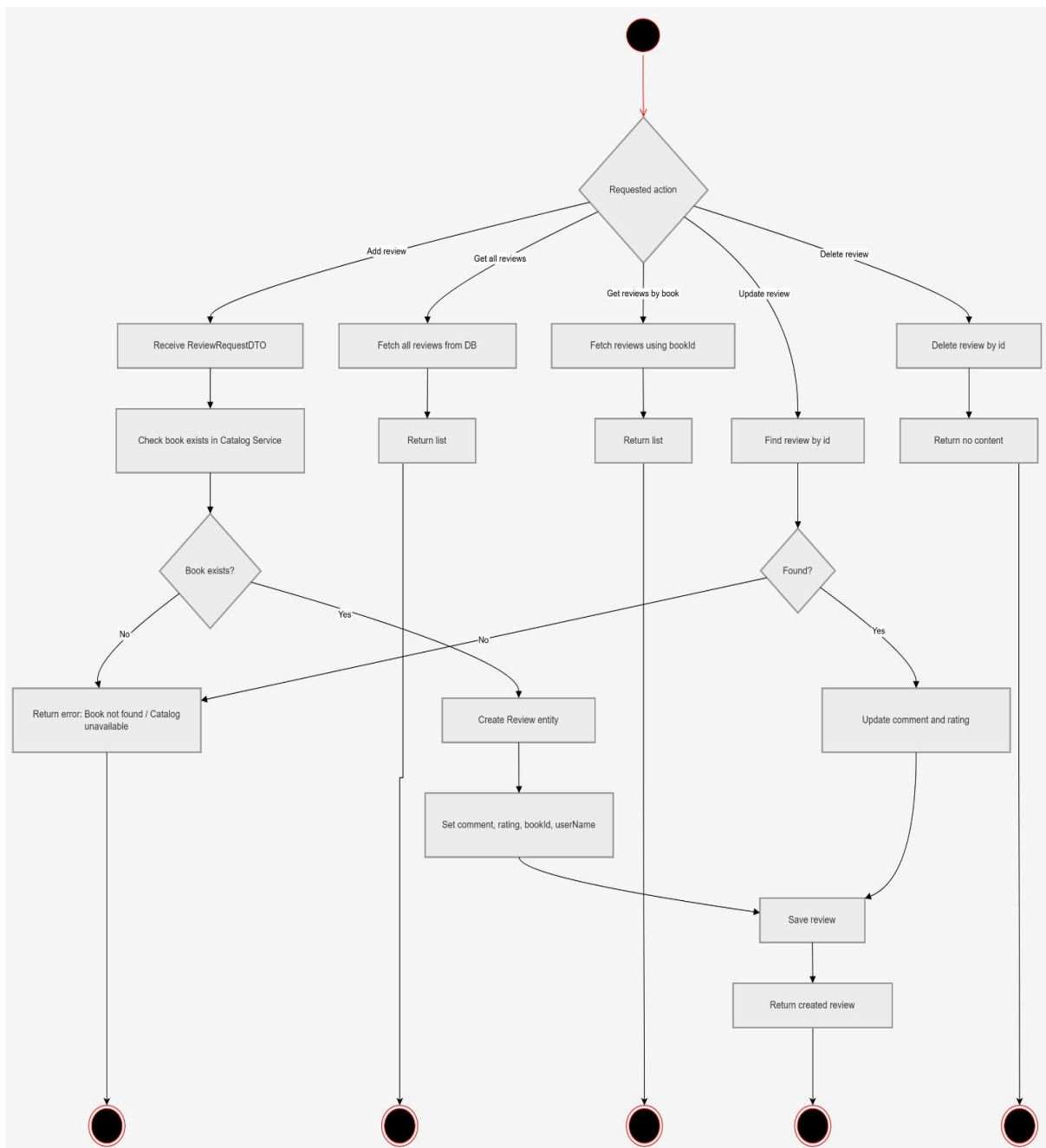
4) Order Service



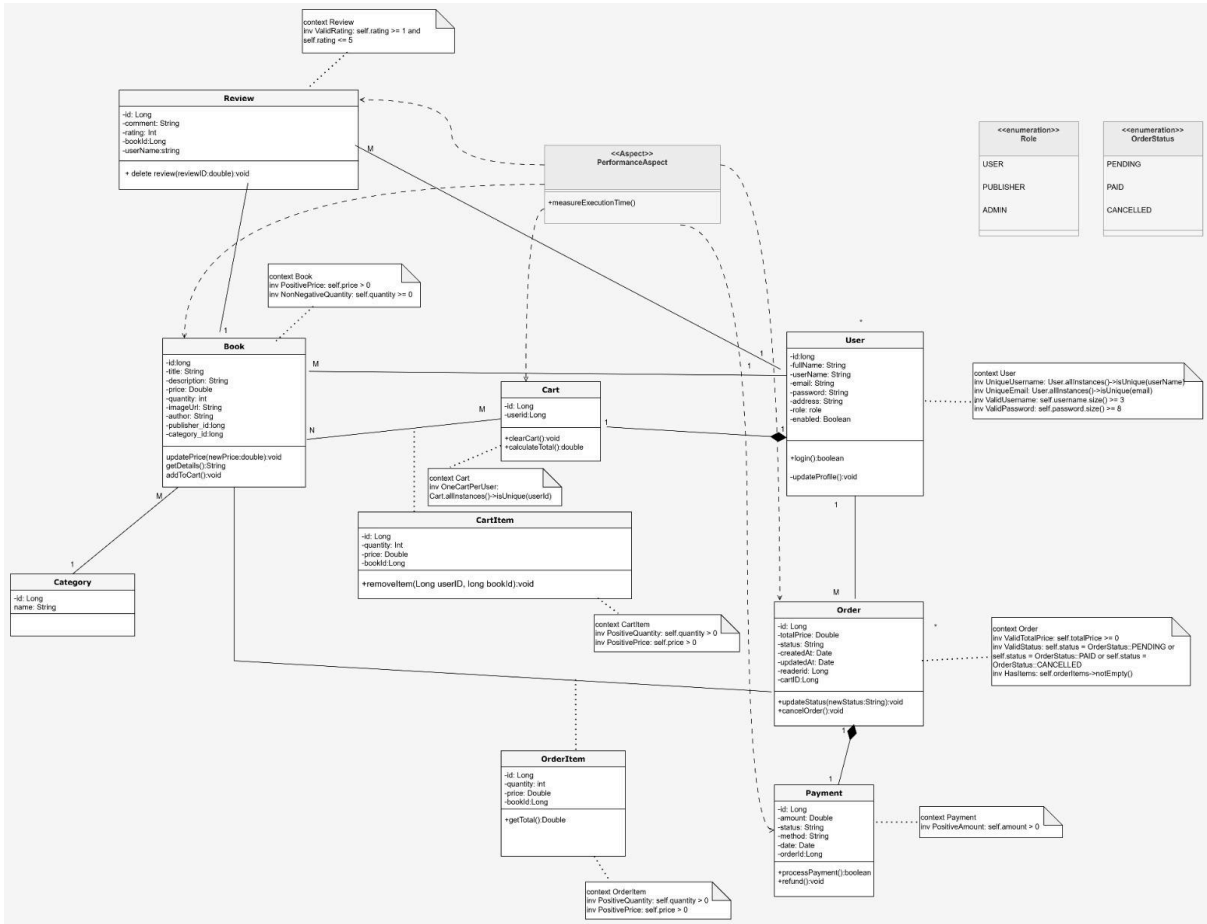
5) Payment Service



6) Review Service



Class Diagram



Use Case Diagram

